Lab1 : back-propagation

311552055 蕭育泓

1. Introduction

這次的 lab 主要是要建立 2 個 hidden layer 的 neural network，實作的部分主要是利用一種 gradient descent 的方式，叫做 back-propagation，而執行方式可以拆解為 forward pass 和 backward pass，利用這兩個 function 得到的值算出 gradient descent，再配合 learning rate，進而去更新我們的下一次各層中使用的 Weight 數值，重複執行直到我們的 Output layer 中的 Loss 收斂。
以下的實作只使用到 numpy、matplotlib。

2. Experiment setups

A. Sigmoid functions

也就是我們所使用的 activation function，這部分是利用 spec 中所給的兩個 def function 來執行(sigmoid、derivative_sigmoid)，主要是在 forward pass 時，各層 W 內積各層的 input 後，要做一次 sigmoid，去當作下一層的 input。而 derivative_sigmoid 是在計算 backward pass 時所需要做的。

```python
def sigmoid(x):
    return 1.0/(1.0 + np.exp(-x))


def derivative_sigmoid(x):
    return np.multiply(x, 1 - x)
```

```python
# backward_pass
derivative_loss = derivative_MSE(y[x_num], a0_Z)
derivative_sigmoid_hidden0_Z = derivative_sigmoid(a0_Z)
hidden0_dc_dz = derivative_sigmoid_hidden0_Z * derivative_loss

hidden2_dc_dz = hidden2_W.T @ hidden0_dc_dz
hidden2_dc_dz *= derivative_sigmoid(a2_Z)

hidden1_dc_dz = hidden1_W.T @ hidden2_dc_dz
hidden1_dc_dz *= derivative_sigmoid(a1_Z)
```

```python
forward_pass(X, init_W, hidden1_W, hidden2_W, hidden1_Z, hidden2_Z, hidden0_Z, a1_Z, a2_Z, a0_Z):
hidden1_Z = init_W @ X
a1_Z = sigmoid(hidden1_Z)
hidden2_Z = hidden1_W @ a1_Z
a2_Z = sigmoid(hidden2_Z)
hidden0_Z = hidden2_W @ a2_Z
a0_Z = sigmoid(hidden0_Z)
# print(a0_Z)
```

B. Neural network

由於這次 lab 所要求的是 2 層的 hidden layer，所以總共會有的 layer 就會有 1 層 Input layer、2 層 hidden layer、1 層 Output layer，而由於 layer 是固定的，所以我把實作 Neural network 所需要使用到的 numpy array 都設為固定 size，總共會使用到的有以下幾個 array :

Init_W= 為 Input layer 與 hidden layer 1 之間的 Weight
hidden1_W= 為 hidden layer 2 與 hidden layer 1 之間的 Weight
hidden2_W= 為 hidden layer 2 與 Output layer 之間的 Weight
hidden1_Z= 為 hidden layer 1，利用 Init_W 和 input data 內積的結果
hidden2_Z= 為 hidden layer 2，利用 hidden1_W 和 a1_Z 內積的結果
hiddenO_Z= 為 Output layer，利用 hidden2_W 和 a2_Z 內積的結果
a1_Z= 為 hidden layer 1，用 hidden1_Z 做 Sigmoid 視為 hidden layer 2 的輸入
a2_Z= 為 hidden layer 2，用 hidden2_Z 做 Sigmoid 視為 Output layer 的輸入
aO_Z= 為 Output layer，用 hidden1_Z 做 Sigmoid 視為 Output 的 Y (pred_y)
hidden1_dc_dz= 為 hidden layer 1，backward pass 中 hidden layer 1 的 result
hidden2_dc_dz= 為 hidden layer 2，backward pass 中 hidden layer 2 的 result
hiddenO_dc_dz= 為 Output layer，backward pass 中 Output layer 的 result
以下是我所設立的 size 大小 ：

```
learning_rate = 0.01
init_W = np.random.uniform(0, 1, size=(4, 2))
hidden1_W = np.random.uniform(0, 1, size=(4, 4))
hidden2_W = np.random.uniform(0, 1, size=(1, 4))
hidden1_Z = np.zeros((4, 1))
hidden2_Z = np.zeros((4, 1))
hiddenO_Z = np.zeros((1, 1))
a1_Z = np.zeros((4, 1))
a2_Z = np.zeros((4, 1))
aO_Z = np.zeros((1, 1))
hidden1_dc_dz = np.zeros((4, 1))
hidden2_dc_dz = np.zeros((4, 1))
hiddenO_dc_dz = np.zeros((1, 1))
```

利用這些 array 來實作我們各層的 neural network。

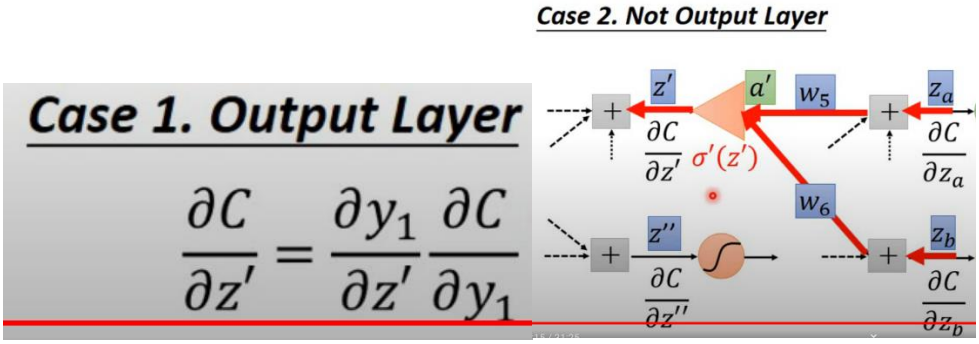
C. Backpropagation

主要利用兩個步驟，分別是 forward pass 與 backward pass，而 forward pass 是會由 input layer 往 output layer 方向去做，主要是為了計算出 a1_Z、a2_Z、aO_Z，去做 backward pass 和 update Weight(細部上方 part B.有寫)，下方是程式碼 ：

```
forward_pass(X, init_W, hidden1_W, hidden2_W, hidden1_Z, hidden2_Z, hidden0_Z, a1_Z, a2_Z, a0_Z):
hidden1_Z = init_W @ X
a1_Z = sigmoid(hidden1_Z)
hidden2_Z = hidden1_W @ a1_Z
a2_Z = sigmoid(hidden2_Z)
hidden0_Z = hidden2_W @ a2_Z
a0_Z = sigmoid(hidden0_Z)
# print(a0_Z)
```

而 backward pass 是會由 output layer 往 input layer 方向去做，主要是去求出 hidden1_dc_dz、hidden2_dc_dz、hidden0_dc_dz(細部上方 part B.有寫)，而比較不一樣的點是在於 output layer 往前那一層我們會利用到 MSE 的微分去求出 derivative_MSE，利用他配合 derivative_sigmoid 求得 hidden0_dc_dz，其餘的都是利用 Weight 與後層的 dc/dz 配合 derivative_sigmoid 來做，下方是程式碼與公式 ：

```
# backward_pass
derivative_loss = derivative_MSE(y[x_num], a0_Z)
derivative_sigmoid_hidden0_Z = derivative_sigmoid(a0_Z)
hidden0_dc_dz = derivative_sigmoid_hidden0_Z * derivative_loss

hidden2_dc_dz = hidden2_W.T @ hidden0_dc_dz
hidden2_dc_dz *= derivative_sigmoid(a2_Z)

hidden1_dc_dz = hidden1_W.T @ hidden2_dc_dz
hidden1_dc_dz *= derivative_sigmoid(a1_Z)
```



最終，我們會知道 hidden1_dc_dz、hidden2_dc_dz、hidden0_dc_dz 和 Input data、a1_Z、a2_Z 與自己先設的 learning rate，利用他們去更新 Init_W、hidden1_W、hidden2_W 做 update Weight，以下是程式碼與公式 ：

$$\theta^1 = \theta^0 - \eta \nabla L(\theta^0)$$
$$\theta^2 = \theta^1 - \eta \nabla L(\theta^1)$$

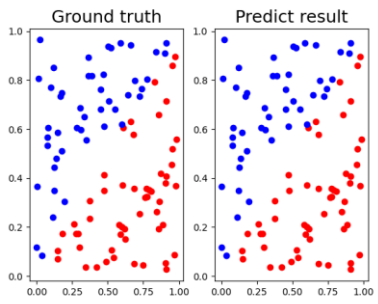
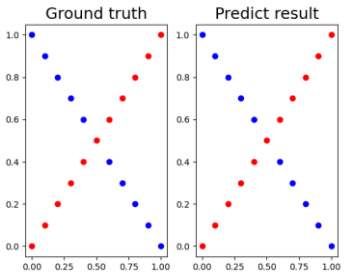
```
# update
init_W -= learning_rate * (hidden1_dc_dz @ X.T)
hidden1_W -= learning_rate * (hidden2_dc_dz @ a1_Z.T)
hidden2_W -= learning_rate * (hidden0_dc_dz @ a2_Z.T)
```

p.s. 我在實作時，我是把每個 input 都做一次 backpropagation，也就是說我每次的 epoch 都會執行一個 x.shape[0]的迴圈，等做完這個迴圈後我才執行我的 testing。以下是我整個 training 的程式碼 ：

```python
# training
for i in range(epoch):
    # loss = 0

    for x_num in range(x.shape[0]):
        X = x[x_num].reshape(2, 1)
        # forward_pass
        init_W, hidden1_W, hidden2_W, hidden1_Z, hidden2_Z, hidden0_Z, a1_Z, a2_Z, a0_Z = forward_pass(
            X, init_W, hidden1_W, hidden2_W, hidden1_Z, hidden2_Z, hidden0_Z, a1_Z, a2_Z, a0_Z)

        # backward_pass
        derivative_loss = derivative_MSE(y[x_num], a0_Z)
        derivative_sigmoid_hidden0_Z = derivative_sigmoid(a0_Z)
        hidden0_dc_dz = derivative_sigmoid_hidden0_Z * derivative_loss

        hidden2_dc_dz = hidden2_W.T @ hidden0_dc_dz
        hidden2_dc_dz *= derivative_sigmoid(a2_Z)

        hidden1_dc_dz = hidden1_W.T @ hidden2_dc_dz
        hidden1_dc_dz *= derivative_sigmoid(a1_Z)

        # update
        init_W -= learning_rate * (hidden1_dc_dz @ X.T)
        hidden1_W -= learning_rate * (hidden2_dc_dz @ a1_Z.T)
        hidden2_W -= learning_rate * (hidden0_dc_dz @ a2_Z.T)

    y_pred = []
    for n in range(x.shape[0]):
        X = x[n].reshape(2, 1)
        init_W, hidden1_W, hidden2_W, hidden1_Z, hidden2_Z, hidden0_Z, a1_Z, a2_Z, a0_Z = forward_pass(
            X, init_W, hidden1_W, hidden2_W, hidden1_Z, hidden2_Z, hidden0_Z, a1_Z, a2_Z, a0_Z)
        y_pred.append(a0_Z[0][0])
    y_pred = np.array(y_pred).reshape(y.shape)
    loss = MSE(y, y_pred)
    if i % print_time == 0:
        print(f'epoch {i}: loss : {loss}')
        learning_array.append(i)
        loss_array.append(loss)

    if i == epoch-1:
        test_y_pred = y_pred
```

3. Results of your testing

A. Screenshot and comparison figure

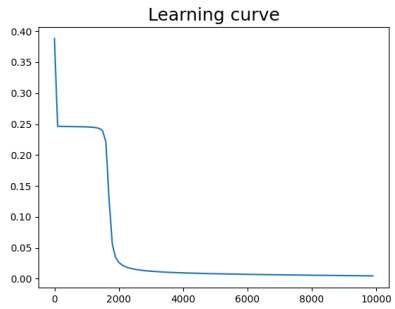
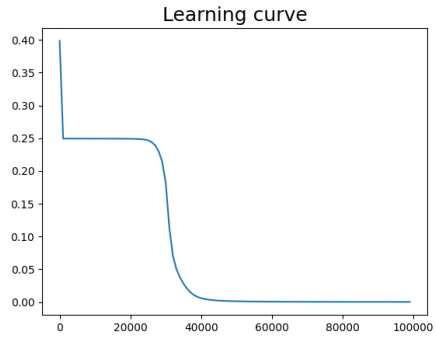
Linear	XOR
	

Linear 與 XOR 的 Predict 皆與 Ground truth 一樣!!!

B. Show the accuracy of your prediction

Linear	XOR
[9.99957207e-01] [8.44141448e-05] [9.99953122e-01] [9.99954178e-01] [9.99956584e-01] [9.97977465e-01] [1.64279182e-03] [1.74046290e-04] [9.99956484e-01]] accuracy : 100.0%	[6.83079817e-03] [9.99716355e-01] [3.71115209e-03] [9.99856647e-01] [2.17727779e-03] [9.99875248e-01] [1.40696602e-03] [9.99881733e-01]] accuracy : 100.0%

C. Learning curve

Linear	XOR
	

X 軸 – MSE

Y 軸 – Epoch

D. Anything you want to share

以下是我 testing 的程式碼，用來求出 accuracy 和 y_pred 的結果 ：

```
# testing
test_label = []
for i in range(test_y_pred.shape[0]):
    if test_y_pred[i] >= 0.5:
        test_label.append(1)
    else:
        test_label.append(0)
correct_count = 0
for i in range(len(test_label)):
    if test_label[i] == y[i]:
        correct_count += 1
print(test_y_pred)
print(f'accuracy : {(correct_count/len(test_label))*100}%')
```

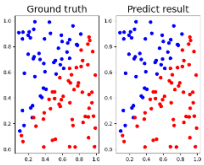
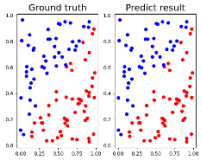
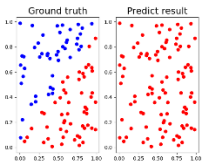
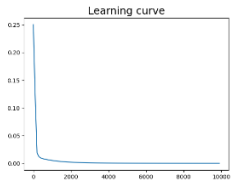
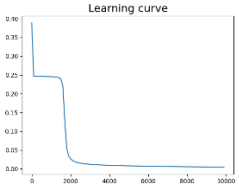
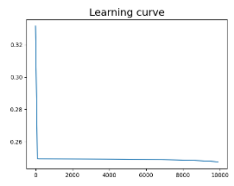
以下是 epoch 對應到的 loss(MSE)

Linear	XOR
epoch 900: loss : 0.24562739388638546 epoch 1000: loss : 0.2454608514395716 epoch 1100: loss : 0.24522641080810037 epoch 1200: loss : 0.24486073505128303 epoch 1300: loss : 0.24421063180854446 epoch 1400: loss : 0.24282698188600865 epoch 1500: loss : 0.23892812229703697 epoch 1600: loss : 0.2209771459068886 epoch 1700: loss : 0.12732638907839747 epoch 1800: loss : 0.05554590155962785 epoch 1900: loss : 0.03472273246766308 epoch 2000: loss : 0.02652022418580128 epoch 2100: loss : 0.022220195316174696 epoch 2200: loss : 0.019545127103981354 epoch 2300: loss : 0.017697104394718735 epoch 2400: loss : 0.01632919829957689	epoch 23000: loss : 0.24855444718231925 epoch 24000: loss : 0.24799023895461378 epoch 25000: loss : 0.2468187981734697 epoch 26000: loss : 0.24427439498217582 epoch 27000: loss : 0.23916691852184807 epoch 28000: loss : 0.22999200883576498 epoch 29000: loss : 0.21421722227959156 epoch 30000: loss : 0.18171453606753837 epoch 31000: loss : 0.11452356552428739 epoch 32000: loss : 0.07126050894104081 epoch 33000: loss : 0.050479172909094094 epoch 34000: loss : 0.037841485673250706 epoch 35000: loss : 0.02826268466014897 epoch 36000: loss : 0.02055217168918964 epoch 37000: loss : 0.014691343748889519 epoch 38000: loss : 0.01055704269423562 epoch 39000: loss : 0.007757060693722264

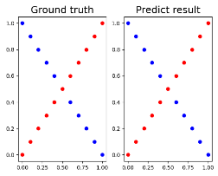
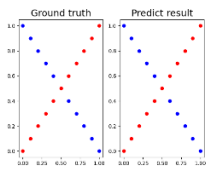
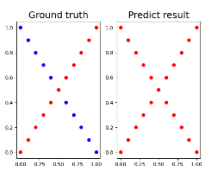
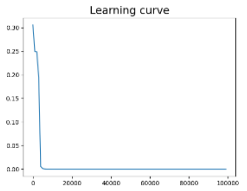
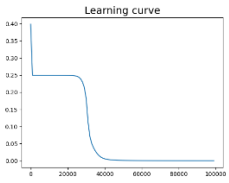
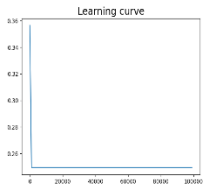
4. Discussion

A. Try different learning rates(LR)

Linear

	LR = 0.1	LR = 0.01	LR = 0.001
comparison figure			
accuracy	[9.99999170e-01] [3.63183232e-06] [9.99999987e-01] [4.05815316e-08] [1.85086970e-08] [9.99999992e-01] [2.92099411e-08] [3.98060768e-08] [3.36937843e-02]] accuracy : 100.0%	[9.99957207e-01] [8.44141448e-05] [9.99953122e-01] [9.99954178e-01] [9.99956584e-01] [9.97977465e-01] [1.64279182e-03] [1.74046290e-04] [9.99956484e-01]] accuracy : 100.0%	[0.4727935] [0.47082064] [0.47545753] [0.47109672] [0.4727288] [0.47494002] [0.4752738] [0.47641095]] accuracy : 53.0%
Learning curve			
loss(MSE)	epoch 0: loss : 0.2502747029649986 epoch 100: loss : 0.12290343432183232 epoch 200: loss : 0.018708190247663035 epoch 300: loss : 0.012009084149305104 epoch 400: loss : 0.009683092171328837 epoch 500: loss : 0.008480382369664134 epoch 600: loss : 0.007659627151747414 epoch 700: loss : 0.006975515120633153 5 epoch 800: loss : 0.006349577300772343 epoch 900: loss : 0.005766326677380 94 epoch 1000: loss : 0.005229225142813762 epoch 1100: loss : 0.004742436251777 82 epoch 1200: loss : 0.004306114518106418	epoch 900: loss : 0.24562739388638546 epoch 1000: loss : 0.2454608514395716 epoch 1100: loss : 0.24522641080810037 epoch 1200: loss : 0.24486073505128303 epoch 1300: loss : 0.24421063180854446 epoch 1400: loss : 0.24282698188600865 epoch 1500: loss : 0.23892812229703697 epoch 1600: loss : 0.2209771459068886 epoch 1700: loss : 0.12732638907839747 epoch 1800: loss : 0.05554590155962785 epoch 1900: loss : 0.03472273246766308 epoch 2000: loss : 0.02652022418580128 epoch 2100: loss : 0.022220195316174696 epoch 2200: loss : 0.019545127103981354 epoch 2300: loss : 0.017697104394718735 epoch 2400: loss : 0.01632919829957689	epoch 9000: loss : 0.2481625676549291 4 epoch 9100: loss : 0.2481062577280201 epoch 9200: loss : 0.2480462251407358 epoch 9300: loss : 0.2479821356475793 7 epoch 9400: loss : 0.2479136169651080 5 epoch 9500: loss : 0.2478402534750700 8 epoch 9600: loss : 0.2477615800505334 2 epoch 9700: loss : 0.2476770748363097 epoch 9800: loss : 0.2475861507779956 2 epoch 9900: loss : 0.2474881456478842 4

XOR

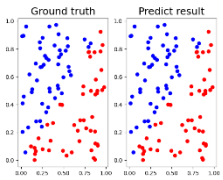
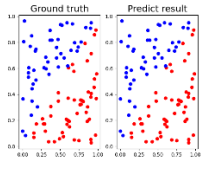
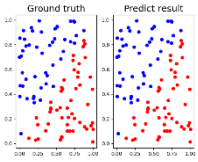
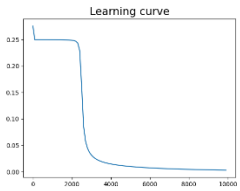
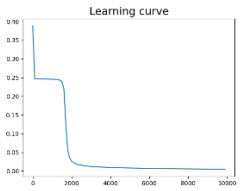
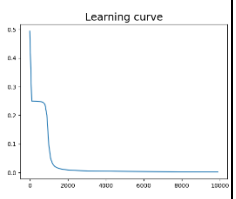
	LR = 0.1	LR = 0.01	LR = 0.001
comparison figure			
accuracy	[9.94449797e-01] [1.09017940e-03] [9.99964415e-01] [5.55059078e-04] [9.99977995e-01] [3.10470144e-04] [9.99979388e-01] [1.94130879e-04] [9.99979747e-01]] accuracy : 100.0%	[6.83079817e-03] [9.99716355e-01] [3.71115209e-03] [9.99856647e-01] [2.17727779e-03] [9.99875248e-01] [1.40696602e-03] [9.99881733e-01]] accuracy : 100.0%	[0.47653833] [0.47658184] [0.47642214] [0.47651064] [0.47630588] [0.47644042] [0.47619005] [0.47637126] [0.47607512] [0.47630326]] accuracy : 52.38095238095239%
Learning curve			
loss(MSE)	epoch 0: loss : 0.3056144441757066 epoch 1000: loss : 0.24939780877376674 epoch 2000: loss : 0.24855611918489331 epoch 3000: loss : 0.19439037626359643 epoch 4000: loss : 0.00575849972180525 epoch 5000: loss : 0.0010784142419984718 epoch 6000: loss : 0.0005104926125762159 epoch 7000: loss : 0.0003185315447396749 6 epoch 8000: loss : 0.0002262376679959496 3 epoch 9000: loss : 0.0001731380228430232 7	epoch 23000: loss : 0.2485544471823192 5 epoch 24000: loss : 0.2479902389546137 8 epoch 25000: loss : 0.2468187981734697 epoch 26000: loss : 0.2442743949821758 2 epoch 27000: loss : 0.2391669185218480 7 epoch 28000: loss : 0.2299920088357649 8 epoch 29000: loss : 0.2142172222795915 6 epoch 30000: loss : 0.1817145360675383 7 epoch 31000: loss : 0.1145235655242873 9 epoch 32000: loss : 0.0712605089410408 1 epoch 33000: loss : 0.0504791729090940 94 epoch 34000: loss : 0.0378414856732507 06 epoch 35000: loss : 0.0282626846601489 7 epoch 36000: loss : 0.0205521716891896 4 epoch 37000: loss : 0.0146913434788895 19 epoch 38000: loss : 0.0105570426942356 2 epoch 39000: loss : 0.0077570606937222 64	epoch 88000: loss : 0.2494362524783011 epoch 89000: loss : 0.2494356165400337 4 epoch 90000: loss : 0.2494349799665272 8 epoch 91000: loss : 0.2494343426163159 2 epoch 92000: loss : 0.2494337043475567 4 epoch 93000: loss : 0.2494330650179549 5 epoch 94000: loss : 0.2494324244846892 2 epoch 95000: loss : 0.2494317826043360 2 epoch 96000: loss : 0.2494311392327943 4 epoch 97000: loss : 0.2494304942252085 epoch 98000: loss : 0.2494298474358912 2 epoch 99000: loss : 0.2494291987182458

以上幾點就可以觀察到幾個項目 ：

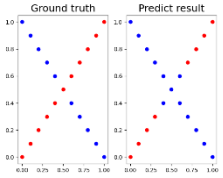
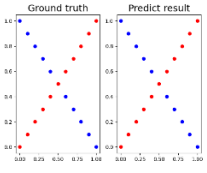
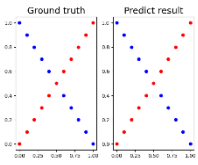
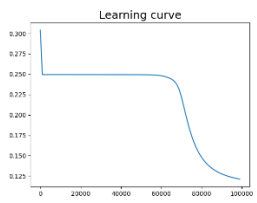
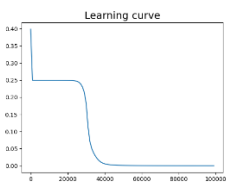
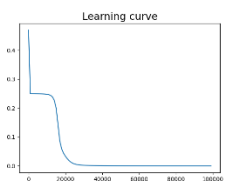
1. LR 越大收斂越快
2. LR 越小收斂後的精準度越高
3. LR 太小導致在 Linear 與 XOR 相同 epoch 收斂所需要花的時間過多導致精準度掰掰

B. Try different numbers of hidden units

Linear

	units = 2	units = 4	units = 8
comparison figure			
accuracy	[9.22836824e-01] [1.19475960e-01] [9.99940108e-01] [2.97912352e-05] [1.76698616e-04] [9.99939185e-01] [9.99697173e-01] [9.99934201e-01] [9.99096099e-01] [9.99937580e-01] [3.19993177e-05]] accuracy : 100.0%	[9.99957207e-01] [8.44141448e-05] [9.99953122e-01] [9.99954178e-01] [9.99956584e-01] [9.97977465e-01] [1.64279182e-03] [1.74046290e-04] [9.99956484e-01]] accuracy : 100.0%	[9.48773030e-01] [4.98102946e-04] [7.11613751e-04] [2.26473718e-04] [2.62529245e-04] [5.48549679e-04] [3.46399861e-04] [6.59037208e-01] [6.86712910e-04] [9.99719267e-01] [9.98361640e-01] [9.98438632e-01]] accuracy : 100.0%
Learning curve			

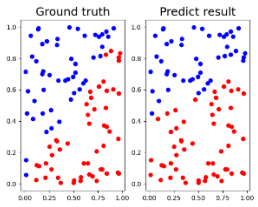
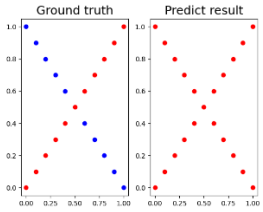
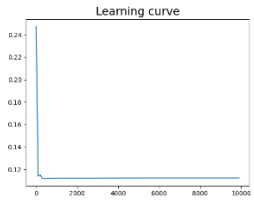
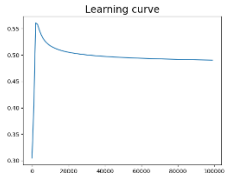
XOR

	units = 2	units = 4	units = 8
comparison figure			
accuracy	[7.16596344e-01] [7.37268761e-01] [6.56476000e-01] [7.56824805e-01] [4.46373686e-01] [7.75250864e-01] [2.00010703e-01] [7.92548031e-01] [6.21058518e-02] [8.08730206e-01] [1.68267169e-02] [8.23822113e-01]] accuracy : 85.71428571428571%	[6.83079817e-03] [9.99716355e-01] [3.71115209e-03] [9.99856647e-01] [2.17727779e-03] [9.99875248e-01] [1.40696602e-03] [9.99881733e-01]] accuracy : 100.0%	[9.76581407e-01] [1.79177694e-02] [1.13760901e-02] [9.78360313e-01] [6.01091441e-03] [9.99923304e-01] [3.09724717e-03] [9.99959650e-01] [1.69739698e-03] [9.99961211e-01] [1.02227189e-03] [9.99960800e-01]] accuracy : 100.0%
Learning curve			

以上幾點就可以觀察到幾個項目 ：

1. hidden units 小，相同 epoch 可能讓 accuracy 下降，就是收斂時間變長
2. hidden units 大，收斂時間快
3. hidden units 大，效果較好

C. Try without activation functions

hidden units=2 LR = 0.01	**Linear**	**XOR**
comparison figure		
accuracy	[0.87940697] [0.99569908] [-0.08707533] [0.81120495] [0.80790736] [0.19185539] [0.98414064] [0.6758606] [-0.30976511] [1.05740294] [0.26254341]] accuracy : 86.0%	[-0.01740047] [-0.01462258] [-0.02030054] [-0.01474478] [-0.02320062] [-0.01486697] [-0.0261007] [-0.01498917] [-0.02900078] [-0.01511136]] accuracy : 52.38095238095239%
Learning curve		
loss(MSE)	epoch 8900: loss : 0.11245710250196864 epoch 9000: loss : 0.11245698134787521 epoch 9100: loss : 0.11245685482554009 epoch 9200: loss : 0.11245672350391477 epoch 9300: loss : 0.11245658790736114 epoch 9400: loss : 0.11245644851936279 epoch 9500: loss : 0.11245630578589645 epoch 9600: loss : 0.11245616011849675 epoch 9700: loss : 0.11245601189704534 epoch 9800: loss : 0.11245586147231197 epoch 9900: loss : 0.11245570916827145	epoch 90000: loss : 0.4909215835889507 epoch 91000: loss : 0.4908502136475551 epoch 92000: loss : 0.4907799586621049 epoch 93000: loss : 0.49071078881888364 epoch 94000: loss : 0.4906426754226725 epoch 95000: loss : 0.4905755908430597 epoch 96000: loss : 0.49050950846386904 epoch 97000: loss : 0.4904444026355312 epoch 98000: loss : 0.4903802486301119 epoch 99000: loss : 0.4903170225989261

以上幾點就可以觀察到幾個項目 ：

1. 沒有 activation functions 後，與之前相同 epoch，ACC 下降嚴重
2. 沒有 activation functions 後，LOSS 觀察出，收斂超級慢